

ЛР4: Развёртывание приложения на удалённом сервере

Студент: Зеленков Евгений

Группа: БР2.2

Тема: развёртывание контейнеризованного backend-приложения на VPS

Цель работы

Цель работы - подготовить удалённый сервер, развернуть на нём приложение из ЛР3 и настроить nginx как обратный прокси для внешнего доступа к API.

Сервер

Для развёртывания использовался VPS:

Параметр

Значение

IP

84.38.180.240

Домен

bk.lab1.zelenkov-labs.ru

Директория приложения

/opt/zelenkov-labs/booking-microservices-lab4

Внешний URL

https://bk.lab1.zelenkov-labs.ru

На сервере уже был установлен nginx и выпущен TLS-сертификат для домена

bk.lab1.zelenkov-labs.ru.

Подготовка сервера

На сервер были установлены Docker и Docker Compose:

```
apt-get update
```

```
apt-get install -y docker.io docker-compose
```

```
systemctl enable --now docker
```

Проверка установленных версий:

Docker version 29.1.3

docker-compose version 1.29.2

Старая версия приложения, запущенная через systemd unit booking-api.service, была остановлена и отключена после успешного запуска контейнерной версии:

systemctl disable --now booking-api.service

Docker Compose для VPS

Для VPS используется отдельный compose-файл:

labs/lab4/deploy/docker-
compose.vps.yml

Он совместим с установленным на сервере docker-compose v1 и отличается от локального compose из ЛР3:

использует формат version: "2.4";

не использует profiles;

публикует наружу только gateway;

PostgreSQL, RabbitMQ и внутренние backend-сервисы доступны только внутри Docker-сети;

gateway опубликован только на loopback-интерфейсе сервера: 127.0.0.1:3105:3105;

внешний HTTPS-доступ к gateway идёт через nginx.

Основные контейнеры:

Контейнер

Назначение

lab4-api-gateway

HTTP API gateway

lab4-identity-service

пользователи и авторизация

lab4-catalog-service

каталог ресторанов и RabbitMQ consumer

lab4-menu-service

меню

lab4-reservation-service

бронирования

lab4-review-service

отзывы

lab4-booking-postgres

PostgreSQL

lab4-booking-rabbitmq

RabbitMQ

Развёртывание приложения

Приложение было скопировано в:

/opt/zelenkov-labs/booking-microservices-lab4

Команды запуска:

`cd /opt/zelenkov-labs/booking-microservices-lab4`

`docker-compose config`

`docker-compose build`

`docker-compose up -d`

Начальные данные были загружены отдельным запуском seed-образа после того, как сервисы стали healthy:

`docker run --rm \`

`--network booking-microservices-lab4_default \`

`-e NODE_ENV=development \`

`-e JWT_SECRET=dev_secret \`

`-e SERVICE_TOKEN=dev_service_token \`

`-e DATABASE_SYNCHRONIZE=true \`

`-e`

`IDENTITY_DATABASE_URL=postgres://booking_user:booking_password@postgres:5432/identity_db \`

`-e`

`CATALOG_DATABASE_URL=postgres://booking_user:booking_password@postgres:5432/restaurant_catalog_db \`

`-e`

`MENU_DATABASE_URL=postgres://booking_user:booking_password@postgres:5432/menu_db \`

`-e`

`RESERVATION_DATABASE_URL=postgres://booking_user:booking_password@postgres:5432/reservation_db \`

`-e`

`REVIEW_DATABASE_URL=postgres://booking_user:booking_password@postgres:5432/review_db \`

`booking-microservices-lab4_seed:latest`

Результат:

Seed completed

Настройка nginx

Для домена используется конфигурация:

labs/lab4/deploy/nginx/bk.lab1.zelenkov-labs.ru.conf

HTTP-запросы на порт 80 перенаправляются на HTTPS. HTTPS listener работает на 127.0.0.1:8444, потому что на сервере уже настроена SNI-маршрутизация TLS-трафика. Nginx проксирует приложение на контейнерный gateway:

`proxy_pass http://127.0.0.1:3105;`

Проверка конфигурации:

`nginx -t`

`systemctl reload nginx`

Результат проверки:

nginx: configuration file /etc/nginx/nginx.conf test is successful

Проверка

Проверка состояния контейнеров:

`docker-compose ps`

Все основные контейнеры находятся в состоянии Up (healthy). Gateway слушает только локальный адрес сервера:

`lab4-api-gateway Up (healthy) 127.0.0.1:3105->3105/tcp`

Проверка health endpoint через публичный домен:

`curl https://bk.lab1.zelenkov-labs.ru/health`

Ответ:

```
{"status":"ok"}
```

Проверка API:

`curl https://bk.lab1.zelenkov-labs.ru/api/v1/restaurants`

Ответ содержит seeded restaurant North Table.

Также был проверен полный сценарий:

Авторизация администратора и пользователя.

Создание бронирования.

Перевод бронирования в Confirmed, затем в Completed.

Создание отзыва.

Модерация отзыва администратором.

Публикация события review.verified через RabbitMQ.

Обработка события catalog-service.

Пересчёт рейтинга ресторана.

Результат проверки:

```
{
  "reservationId": "6d3ada6a-2c2f-4f81-abd1-b4d9cbe9d9a6",
  "reviewId": "1696d449-da14-478a-b658-756fbf3c4e79",
  "ratingAfterEvent": 5
}
```

Вывод

В ходе работы VPS был подготовлен для запуска контейнеризованного backend-приложения. Приложение из ЛР3 развёрнуто через Docker Compose, nginx настроен как reverse проху для домена `bk.lab1.zelenkov-labs.ru`, а работоспособность проверена через публичный HTTPS endpoint и полный бизнес-сценарий с RabbitMQ.